

Distributed Metrics Logging and Aggregation System Production

Architecture and Operating Model Date: 2025-08-10 UTC

Distributed Metrics Logging and Aggregation System Production

Architecture and Operating Model Date: 2025-08-10 UTC

One Pager Summary Goal Build a global, multi tenant metrics platform that accepts high cardinality time series from services, devices, and applications, aggregates and stores them efficiently, and serves query and alert workloads at interactive latencies. The system must be horizontally scalable, cost aware, secure, and resilient with clear service level objectives.

Key Outcomes

Reliable ingestion with backpressure and replay and exactly once effective processing. Consistent query semantics with downsampling, rollups, and sparse data handling. Sub second query response for common time ranges and alert evaluation that completes within one evaluation window. Clear multi tenancy with quota, rate limiting, and budget enforcement. Operational excellence with automated deployments, canary and rollback, and deep observability.

Primary Assumptions

Global footprint across multiple regions. Sustained ingestion between one and fifty million samples per second globally depending on adoption. Cardinality in the order of tens to hundreds of millions of active time series. Query mix dominated by recent time ranges such as last fifteen minutes and last one hour with periodic heavy batch jobs for dashboards and reporting. Data retention tiers including hot recent, warm downsampled, and cold archival.

Functional Requirements

Accept metric writes in line protocol and open formats. Support counter, gauge, histogram, summary, and distribution types with native exemplars. Support tags and labels with strict cardinality governance and tenant enforced label constraints. Provide a query language compatible with PromQL and SQL like aggregations for rollups. Expose alerting rules and recording rules with evaluation, deduplication, and silencing. Offer multi region tenancy with per tenant auth, quotas, and usage reporting. Provide administrative tooling for schema governance, label allow lists, and backfills. Provide data export and backfill APIs for recovery and migration scenarios.

Non

Availability of write and read APIs at ninety nine point nine five percent or better per region. Ingest durability with recovery point objective of zero for acknowledged writes and recovery time objective in minutes. Query latency p95 under five hundred milliseconds for common ranges and p99 under one second. End to end alert evaluation latency under the configured evaluation window plus a small constant. Security and compliance aligned with SOC two Type two, GDPR, and CCPA including data minimization and retention controls. Cost efficiency achieved through compression, compaction, and tiered storage.

Architecture Overview

High Level Regional cells ingest, process, store, and serve metrics. A thin global control plane manages tenancy, auth, quotas, schemas, routing, catalog, and fleet configuration. Clients connect to the nearest healthy cell. Cells are independent to limit blast radius.

Ingestion

Edge gateway terminates TLS, authenticates with OAuth or OIDC tokens or signed service credentials, enforces rate limits, and validates metrics against tenant label policies. Producers send batches using HTTP or gRPC. Batches enter a partitioned log such as Kafka keyed by tenant and a hash of series identity. Schema Registry tracks metric families and label schemas and versions. Dead letter topics and quarantine capture validation failures.

Stream Processing

Flink or a Go based shard processor performs deduplication using write ids and per series sequence numbers. Counters are normalized for resets. Rollup calculators compute pre aggregates at fixed resolutions such as ten seconds and one minute. Downsamplers produce minute, five minute, and one hour rollups with mergeable sketches for percentiles. Exemplar links are stored for sampled traces

when provided.

Storage Strategy

Hot Path A write optimised time series database stores recent samples at raw resolution. Options include Mimir or Cortex with object storage blocks and a short term cache, or ClickHouse with wide table schemas, or a custom LSM based shard store. Redis or Memcached provide short lived ingestion caches for out of order handling windows. Warm and Cold Object storage holds compressed blocks in columnar formats with index metadata, using Iceberg or Delta for metadata management and versioning. A compactor merges small blocks, deduplicates overlaps, and builds downsampled tiers. Index A series index maps metric names and label sets to shard locations. This index is partitioned by tenant and metric family and held partly in memory and partly on SSD. Metadata A catalog stores metric definitions, retention policies, label constraints, and ownership.

Query Layer Query frontends accept PromQL and SQL style aggregation. They perform query sharding by time range and tenant and route to the appropriate backends. Query backends read from hot stores and object storage with caching. They perform partial aggregations and stream results to the frontend. Result caches store recent query fragments. A rule engine executes alerting and recording rules by using the same query path with guaranteed isolation and scheduling fairness.

Scaling Approach Kafka topics partition by tenant and a stable hash of series identity to preserve per series ordering while enabling parallelism. Processors scale linearly with partitions. TSDB backends shard by tenant and time range and optionally by metric family to avoid hot shards. Object storage scales with compaction and partition evolution. Frontends and backends scale independently with autoscaling based on tail latency, queue depth, and fetch throughput. Cross region architectures rely on per region cells and a global catalog. Cross region queries either federate results from multiple regions or use async replication for key tenants.

Data Models

Metric Sample tenant id, metric name, labels as a compact key or map, timestamp, value, exemplar id optional. Series Index tenant id, metric name, label fingerprint, first seen at, last seen at, retention tier, current shard id. Rollup Point tenant id, metric name, label fingerprint, time bucket, aggregate values such as count and sum and min and max and p50 and p90 and p99 via sketches. Alert Rule tenant id, rule id, expression, window, evaluation every, severity, labels, annotations, owner, enabled, last evaluation metadata. Silence tenant id, matcher set, starts at, ends at, creator, reason. Quota tenant id, write per second and series limits and retention policies.

Latency Budgets and Service Levels Write path budget for gateway to Kafka produce under twenty milliseconds p95 within region. Processing budget of two hundred milliseconds p95 for dedupe and rollup of the batch. Hot store write budget of one hundred milliseconds p95. Query budget for frontend and backend including cache under five hundred milliseconds p95 for common time ranges such as last one hour at one minute resolution. Alert evaluation budget equals window length plus up to five hundred milliseconds for query and dispatch. Read API target availability ninety nine point nine five percent. Write API target availability ninety nine point nine five percent.

Key Algorithms and Workflows

Cardinality Control Enforce label allow lists and deny lists at ingest. Apply per tenant series limits and adaptive rejection for explosive labels such as user id or request id. Detect high cardinality via sketches and sampling and notify owners. Deduplication and Exactly Once Effective Semantics Use idempotency keys at batch and sample level. For replicated scrapes or agent retries keep the highest sequence number per series and window. Use compaction to remove overlaps in cold blocks.

Downsampling and Rollups Use tumbling windows for min and max and sum and count and sum of squares. For percentiles use DDSketch or t digest to maintain mergeable summaries. Store both raw and rollup series to serve different latency and cost profiles. Histogram Support Ingest native histogram buckets with counters and convert to quantiles using histogram native queries or sketch based

approximations. Alert Processing Rules are grouped by data locality. The engine executes queries close to data and deduplicates alerts across replicas. Silences and inhibition rules are evaluated before dispatch. Anomaly Detection Optional detectors use statistical bands or seasonal decomposition on rollups. Detectors write suggested alerts that operators can adopt.

Security Reliability and Compliance PII is avoided in metrics. Enforce label policies that block PII and secrets. Encrypt in transit with TLS one point three and at rest with KMS managed keys, optionally per tenant. Authenticate with OAuth and service accounts and short lived tokens and mTLS in the mesh. Authorize with RBAC and ABAC using tenant, team, and environment attributes. Audit logs for configuration and rule changes and access. Multi AZ deployments for all critical tiers. Disaster recovery plans include object storage versioning and index backups. Edge rate limiting and WAF for public endpoints.

Multi Tenancy Tenant id is mandatory in every path. Logical isolation through tenant segmented topics, shards, and indexes. Optional hard isolation with dedicated namespaces and VPCs for large tenants. Per tenant budgets for write rate, series count, storage, and query concurrency with visibility via usage reports. CI CD and Infrastructure as Code Everything as code using Terraform or Pulumi for networks and clusters and IAM and storage. Helm manages services. Argo Rollouts provides canary and progressive delivery based on p95 latency and error rate and Kafka lag. Schema and contract tests run in CI and block promotion on failure. Online migrations and backfills use dual write and shadow read with automatic rollback triggers.

APIs and Integration Points Ingest API accepts batched metric writes and supports compressed payloads. Query API accepts PromQL and SQL like aggregates. Rules API manages alert and recording rules and silences. Metadata API manages metric definitions, label governance, and quotas. Export API streams downsampled data or raw blocks for external use. Agent integrations support Prometheus remote write, OpenMetrics, and OpenTelemetry metrics. Alert delivery integrates with PagerDuty, Slack, email, and webhooks.

Observability and Operations

Self monitoring with golden signals such as ingestion lag and series churn and cache hit ratio and query tail latency and compactor backlog. OpenTelemetry for traces and metrics. Dashboards for budget burn down and tenant usage and cardinality hotspots. SLO based autoscaling for frontends, processors, and compaction jobs. Runbooks for hot shard mitigation, cache warmups, block compaction backlog, and index rebuild. Operational budgets and alerts on object store costs and egress and query scans.

Capacity Planning and Cost Estimates

Reference scale of ten million samples per second sustained with one hundred million active series across three busy regions. Kafka fleet of nine to fifteen brokers per region with replication factor three. Processing pool sized for one hundred to three hundred partitions with checkpoint intervals under five seconds. Hot store sized for eight to twenty four hours of raw retention and high write throughput. Object storage sized for thirty to ninety days of downsampled data and one year of cold archives with compaction jobs. Rough monthly cost order of magnitude between forty thousand and one hundred fifty thousand per busy region depending on retention and query profile.

Testing Strategy and Failure Drills

Unit tests for parsing and validation and counter normalization. Property tests for dedupe and rollups and histogram merging. Contract tests for agent protocols and alert delivery webhooks. Load tests for high write rates and query fanout and hot shard scenarios. Soak tests for compaction and small file growth and index churn. Chaos drills for broker loss, cache flush, compactor backlog, and regional isolation with federated queries. Disaster recovery rehearsals validate restore and

replay time for index and object storage.

Phased Build Plan

MVP three months for single region cell, Prometheus remote write compatibility, raw to bronze ingestion with Kafka, hot store with twelve hour retention, basic query and alerting, and minimal downsampling. Phase two six to nine months for multi region cells, rollups and sketches, stronger index and caching, rule federation, and SLO based autoscaling. Phase three twelve months for hard isolation options, advanced anomaly detectors, cost controls with aggressive tiering, and cross region query federation with a global catalog.

Key Choices Trade Offs Risks and Mitigations Adopting a lakehouse like object storage for durability lowers cost but increases read latency for deep history which is mitigated with caches and rollups. Supporting PromQL preserves ecosystem compatibility but constrains some modeling choices which is mitigated by a dual path that also exposes SQL style aggregates. High cardinality can threaten stability which is mitigated with label governance and quotas and proactive detection. Exactly once semantics are difficult across systems which is mitigated by idempotency keys and dedupe and compaction to achieve effective once behavior. Aggressive compression and compaction reduce cost but require careful scheduling to avoid query interference which is mitigated with workload isolation and window based compaction.